Taller 3: Microservicios y eventos

CE5508 · Arquitectura orientada a servicios aplicada a sistemas emergentes · Trabajo en grupo · En clase

Escenario. La plataforma ya tiene frontend, gateway, autenticación con JWT/JWKS y bases de datos separadas. Ahora debe gestionar dispositivos, decidir quién puede acceder a cada uno y reaccionar a eventos sin acoplar todos los servicios por HTTP.

La pregunta. ¿Cómo diseñar un sistema donde un microservicio publique un suceso y otros reaccionen de forma desacoplada, manteniendo autenticación, autorización por recurso y despliegue local en Docker?

Tecnología de eventos

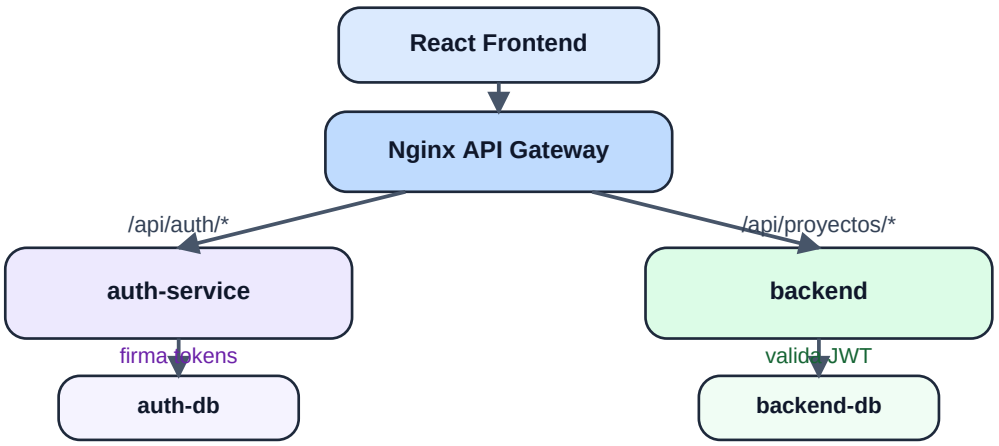
Usen **RabbitMQ** como broker. La comunicación entre productor y consumidor debe ocurrir mediante mensajes.

Ejecución local

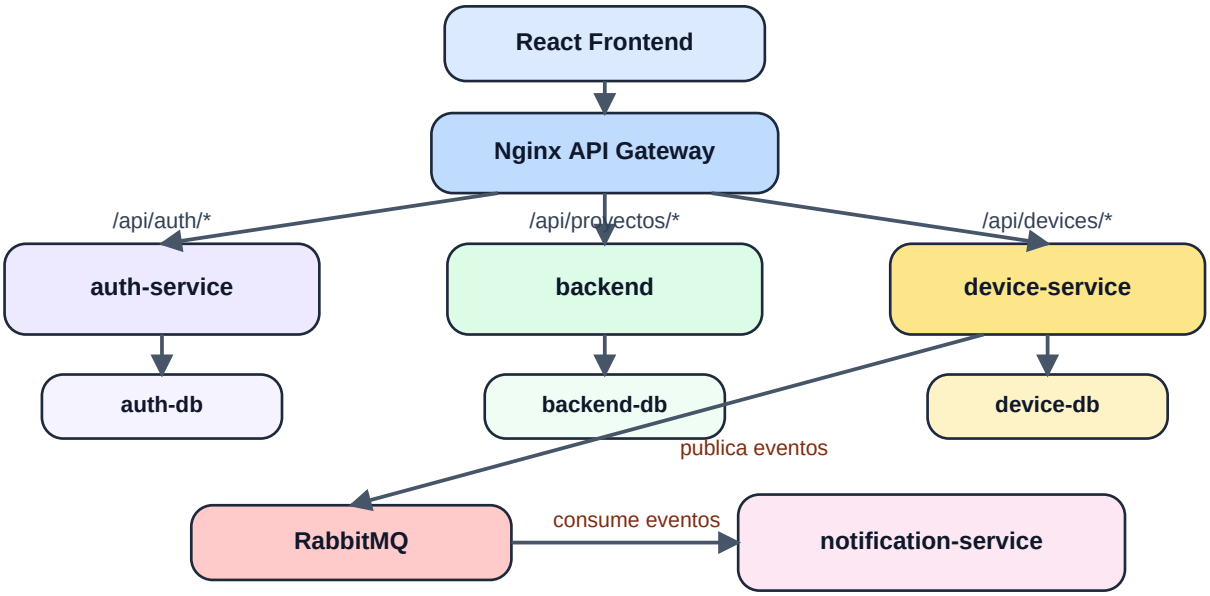
Todo debe correr **localmente** con **Docker Compose**, incluyendo RabbitMQ y todas las bases de datos.

1 · Arquitectura

Arquitectura actual después del Taller 2



Arquitectura objetivo del Taller 3



2 · Lo que deben construir

- **device-service** con su propia base de datos. Debe registrar dispositivos, consultarlos y validar si el usuario autenticado tiene permiso sobre el recurso solicitado.
- **RabbitMQ** como infraestructura de eventos. El productor publica mensajes y los consumidores reaccionan sin depender de llamadas HTTP directas.
- **notification-service** como consumidor independiente. Debe escuchar eventos y registrar una notificación por cada evento procesado.
- La autenticación del **Taller 2** se mantiene. El frontend sigue hablando con **una sola puerta de entrada**.

3 · Contratos mínimos

Ruta	Método	Contrato esperado
/api/devices	POST	{serial, nombre, ownerType, ownerId} → 201
/api/devices/:id	GET	Devuelve el dispositivo si el usuario tiene permiso; 401/403/404 según corresponda
/api/devices/:id/alerts	POST	{level, message} → 202 y publica **device.alert.triggered**

- Eventos mínimos: **device.registered** y **device.alert.triggered**.
- Cada evento debe incluir al menos: **eventId, type, occurredAt, deviceId** y una referencia al usuario o proyecto.
- **notification-service** puede ser interno; no necesita exponer API pública para cumplir el taller.

4 · Requisitos

Requisito 1 — Servicio de dispositivos y autorización por recurso

Agregar **device-service** y **device-db**. Cada dispositivo pertenece a un usuario o proyecto. Validar el JWT no basta: el servicio debe comprobar si ese usuario puede ver o modificar ese dispositivo. Sin token o token inválido → **401**. Identidad válida sin permiso → **403**.

Requisito 2 — Publicación de eventos

Al registrar un dispositivo, publicar **device.registered**. Al reportar una anomalía, publicar **device.alert.triggered**. Cada evento debe incluir al menos: **eventId, type, occurredAt, deviceId** y una referencia al usuario o proyecto.

Requisito 3 — Consumidor independiente

Crear **notification-service** para escuchar la cola y registrar notificaciones. Debe confirmar mensajes procesados y tolerar repetidos usando **eventId**. Si ocurre un error temporal, el mensaje no debe perderse silenciosamente.

Requisito 4 — Despliegue local en Docker

Todos los componentes deben levantarse con **docker compose up --build**. Cada servicio corre en su propio contenedor y con su propia base de datos. No se permite depender de servicios en la nube.

5 · Pruebas obligatorias

- Registrar un dispositivo con JWT válido → el dispositivo se guarda y se publica **device.registered**.
- Consultar un dispositivo ajeno con JWT válido → **403**. Consultar sin JWT → **401**.
- Reportar una anomalía → se publica **device.alert.triggered** y **notification-service** registra la notificación correspondiente.
- Detener **notification-service**, generar un evento y reiniciarlo → el evento pendiente puede procesarse al volver a levantar el servicio.
- Reenviar un mismo **eventId** → no se genera una notificación duplicada.

6 · Entregable

- **Sistema ejecutándose localmente** con Docker Compose.
- **Repositorio actualizado** con los nuevos servicios, configuración y dependencias.
- **Explicación breve** de la arquitectura, del flujo de eventos y de cómo verificaron las cinco pruebas.
- **Herramientas utilizadas**, incluida IA, y qué decisiones técnicas tomaron.

Regla. Pueden usar documentación, librerías e inteligencia artificial. La solución final debe poder explicarse y demostrarse en vivo.

7 · Resumen rápido

Servicio nuevo	device-service y notification-service
Tecnología de eventos	RabbitMQ
Seguridad	Se mantiene JWT/JWKS del Taller 2 y se agrega autorización por recurso
Despliegue	Todo local, en Docker Compose